PanelForge

Product & Technical Design Specification

Version: 1.2 Consolidated Development Baseline
Status: Approved for Development
Initial Platform: Linux Mint
Primary Hardware Target: NVIDIA GTX 1660 Ti class systems
Technology: Electron, TypeScript, Node.js, SQLite, local AI providers

---

1. Purpose and Product Vision

PanelForge is a local-first desktop application for creating illustrated stories, comics, graphic novels, and webtoons using a combination of traditional creative tools and AI-assisted production.

The application should support two primary workflows:

1. **Manuscript-to-comic production:** Import an existing novel, short story, or other written work and transform it into an illustrated narrative.
2. **Original comic creation:** Build a comic or graphic novel from scratch using characters, locations, objects, storyboards, artwork, and page layouts.

PanelForge is intended to be a complete production environment rather than a simple image-generation interface.

The creator remains in control. AI tools assist with analysis, design, composition, and image generation, but the user approves the material that becomes part of the finished work.

The application must prioritize:

- Local-first operation and user ownership of projects.
- Compatibility with modest hardware.
- Independent generation and regeneration of assets.
- Character and visual continuity.
- Transparent AI generation processes.
- Reliable project storage and portability.
- A development process based on small, testable tasks.

Product positioning

A local-first comic studio built for creators, not just people with flagship GPUs.

PanelForge should allow people to work with the hardware they already own. Generation may take longer on modest systems, but the application should make that process manageable through sequential processing, visible progress, resource-aware scheduling, and independently saved results.

The application must never assume that every user has a high-end graphics card.

---

2. Core Design Principles

These principles govern the entire application and should be used when evaluating future feature requests.

2.1 Local-first

Projects and assets should be stored locally by default.

Cloud services may be supported as optional AI providers, but cloud access must never be required to open, edit, or export an existing project.

2.2 User ownership

Users own their manuscripts, characters, generated images, project files, and other creative assets.

PanelForge must not silently replace approved work or make users dependent on a proprietary cloud service.

2.3 Independent regeneration

Every major generated asset must be independently regenerable.

Examples include:

- Character portraits.
- Character reference views.
- Expressions and poses.
- Locations.
- Objects and equipment.
- Storyboard panels.
- Individual artwork panels.

- Individual panel layers or regions.
- Complete comic pages.

Regenerating one asset must not automatically regenerate unrelated assets.

2.4 Preserve approved work

Generated images are new versions, not replacements.

Previous generations remain available. Approved versions are preserved until the user explicitly changes their status or deletes them.

2.5 Hardware accessibility

The application must be designed around realistic hardware limitations.

Small GPU Mode is a core product feature, not an emergency fallback.

2.6 Transparent generation

Users should always be able to understand:

- What the application is doing.
- What is waiting in the queue.
- What resources are being used, when known.
- What has completed.
- What has failed.
- Why a task has paused.
- What can be regenerated independently.

The application must not invent progress percentages or estimated completion times.

2.7 Canon is not speculation

Story analysis must distinguish between:

- Explicit information from the manuscript.
- Reasonable interpretations.
- AI-generated suggestions.
- Unknown or contradictory information.

AI-generated suggestions must never silently become established story facts.

2.8 Small, independently testable development tasks

Development must be divided into tasks that can be completed, tested, and accepted individually.

No task should become a large, loosely defined subsystem that takes weeks to implement before anything can be verified.

---

3. Target Users and Use Cases

PanelForge is intended for:

- Independent authors.
- Comic and graphic novel creators.
- Webtoon creators.
- Writers who want to illustrate existing fiction.
- Artists who want AI-assisted production tools.
- Hobbyists working with local AI models.
- Creators using modest or older computer hardware.

3.1 Hardware assumptions

The initial development environment uses an NVIDIA GTX 1660 Ti, expected to have approximately 6 GB of VRAM, subject to hardware detection.

This is an important design constraint.

The application should support workflows that do not require a flagship GPU. It must accommodate slower generation, sequential processing, smaller image sizes, and optional CPU or remote processing.

PanelForge must not promise that every model or ComfyUI workflow will run on every supported system.

---

4. Product Scope

4.1 Initial product scope

The application will eventually include:

- Project management.
- Manuscript import and analysis.

- Character management and reference generation.
- Location and object management.
- Storyboard creation.
- AI artwork generation.
- Comic page layout.
- Continuity checking.
- Publishing and export.
- Generation history and asset versioning.
- Resource-aware task management.

4.2 Explicit non-goals for the initial release

The following are deferred:

- Animation production.
- Video production.
- Multiplayer collaboration.
- Cloud synchronization.
- Social networking.
- Marketplace functionality.
- Mobile applications.
- Automatic publishing to commercial platforms.
- Advanced LoRA training.
- Complex distributed rendering.

These may be considered later, but they must not delay the initial production workflow.

---

5. Core User Workflow

The intended production workflow is:

1. Create a project.
2. Import a manuscript or start an original story.
3. Analyze the story and extract relevant information.

4. Review characters, locations, objects, and events.
5. Create or approve character reference packages.
6. Establish the project's art style.
7. Break the story into scenes.
8. Create and approve storyboards.
9. Generate individual panels.
10. Review, compare, and approve artwork.
11. Assemble panels into pages.
12. Add dialogue, captions, and sound effects.
13. Run continuity and quality checks.
14. Export the finished work.

The workflow must not be rigid.

Users should be able to work on storyboards or page layouts while unrelated generation tasks are running, subject to available system resources.

Users should also be able to skip or manually perform steps when appropriate.

---

6. Technical Architecture

PanelForge will use a modular architecture with a clear separation between the application interface and its underlying business logic.

6.1 Electron Main Process

Responsibilities:

- Application lifecycle.
- Window management.
- Filesystem access.
- Secure IPC handling.
- Configuration management.
- Logging.
- Database service coordination.
- Operating system integration.

- Task process management.

6.2 Renderer

Responsibilities:

- User interface.
- Project navigation.
- Editing interfaces.
- Character and asset previews.
- Storyboard editing.
- Artwork review.
- Comic layout.
- Task queue visualization.

The renderer must not have unrestricted Node.js or filesystem access.

6.3 Preload Bridge

The preload process provides a restricted, typed API between the renderer and Electron's main process.

Only explicitly approved operations may be exposed.

6.4 Core Engine

The Core Engine contains the application's primary business logic.

It should be designed to operate independently of the renderer.

Responsibilities include:

- Project operations.
- Asset management.
- Task orchestration.
- Story analysis.
- Character and location management.
- Generation coordination.
- Continuity checking.
- Export operations.

The architecture should eventually permit a command-line interface or headless execution environment without rewriting the core application.

6.5 Task Manager

The Task Manager coordinates background operations.

It must support:

- Persistent task records.
- Task priorities.
- Queues.
- Pause and cancellation.
- Retry.
- Dependencies.
- Progress reporting.
- Resource restrictions.
- Partial completion.
- Failure recovery.

6.6 AI Provider Adapters

AI providers must be accessed through independent adapters.

Initial providers:

- LM Studio for local language models.
- ComfyUI for image generation.
- Optional Ollama support.
- Optional cloud AI providers later.

The absence of an available AI provider must not prevent users from opening and editing existing projects.

---

7. Project Storage and Data Portability

Projects must be portable and independent of the machine on which they were created.

7.1 Project structure

A typical project may contain:

```
MyComic/
├── project.json
├── database/
│   └── project.db
├── manuscripts/
│   ├── original/
│   └── processed/
├── characters/
│   ├── robert/
│   │   ├── profile.json
│   │   ├── age-08/
│   │   │   └── references/
│   │   └── age-18/
│   └── ember/
├── locations/
├── objects/
├── styles/
├── storyboards/
├── artwork/
│   ├── generated/
│   └── approved/
├── layouts/
├── exports/
├── workflows/
├── backups/
└── logs/
```

The exact structure may evolve, but the project must remain understandable and portable.

7.2 Path handling

All project asset references must use paths relative to the project root.

Never store machine-specific absolute paths as the canonical location of project assets.

Example:

`characters/robert/age-08/references/front.png`

The application must prevent unsafe path traversal and verify that referenced files remain inside the permitted project structure unless an explicitly authorized external asset reference is used.

7.3 SQLite

SQLite is the initial metadata and relationship database.

Required configuration:

- WAL mode.
- Foreign key enforcement.

- Busy timeout.
- Database migrations.
- Controlled write access.
- Serialized write transactions.
- Retry handling for transient locking errors.
- Regular backups.

Database writes should pass through a controlled service owned by the Core Engine.

The application must avoid uncontrolled concurrent writes from multiple modules.

7.4 Asset registry

Every managed asset should have metadata including:

- Unique identifier.
- Asset type.
- Relative path.
- Creation date.
- Modification date.
- Content hash.
- Version.
- Approval status.
- Relevant relationships.
- Generation provenance, when applicable.

The application should be able to identify missing, moved, or corrupted assets.

7.5 Backups and snapshots

The application must support:

- Project backups.
- Snapshot creation.
- Snapshot restoration.
- Integrity verification.
- Recovery after interrupted operations.

User projects do not require Git.

Git will be used for PanelForge's own source code. Project-level history, snapshots, and branches will be handled internally.

---

8. Subsystem Specifications

8.1 Project Manager

Functions:

- Create projects.
- Open and close projects.
- Rename projects.
- Archive projects.
- Browse recent projects.
- Manage project metadata.
- Associate manuscripts.
- Maintain asset indexes.
- Create backups.
- Verify project integrity.

8.2 Manuscript Manager

Supported import formats:

- TXT.
- Markdown.
- DOCX.
- EPUB.
- PDF.

The original manuscript must remain preserved.

The system should extract:

- Chapters.
- Headings.

- Paragraphs.
- Scene breaks.
- Dialogue.
- Potential character references.
- Potential locations.
- Potential objects.

Every extracted scene should retain a relationship to its original manuscript source.

8.3 Story Analysis Engine

The Story Analysis Engine identifies and organizes story information.

Potential extraction targets:

- Characters.
- Locations.
- Objects.
- Dialogue.
- Events.
- Scene transitions.
- Relationships.
- Chronology.
- Character ages.
- Physical descriptions.
- Clothing.
- Injuries.
- Important visual details.

The system must distinguish confirmed facts from interpretations and suggestions.

Conflicting information should be reported for review rather than silently resolved.

8.4 Character Studio

The Character Studio manages character profiles, reference images, and character versions.

A character profile should support:

- Name.
- Description.
- Physical appearance.
- Age.
- Personality.
- Clothing.
- Equipment.
- Distinguishing features.
- Relationships.
- Timeline.
- Reference images.
- Generation history.
- Approval status.

Generation options should eventually include:

- Portraits.
- Full-body front views.
- Side views.
- Rear views.
- Expressions.
- Poses.
- Clothing variations.
- Equipment sheets.

Character versioning

A character may have multiple distinct visual versions.

For example:

- Robert, age 7.
- Robert, age 8.
- Robert, age 12.
- Robert, age 18.

- Robert, adult.

Each version must be independently identifiable.

A storyboard panel must reference the exact character version it uses.

The system should warn if an incorrect version is assigned to a scene.

Minimum Viable Reference Set

Character reference completeness should be measured and displayed.

The system should support configurable reference tiers:

Basic

- At least one useful visual reference.

Recommended

- Portrait.
- Front view.
- At least one expression.

Full

- Portrait.
- Front, side, and rear views.
- Multiple expressions.
- Useful poses.
- Clothing and equipment references where applicable.

The exact requirements may vary according to character type and intended use.

A reference completeness score should provide useful guidance, not impose arbitrary hard restrictions.

Users must be able to proceed with incomplete reference sets.

Warnings should explain what additional references would improve and why.

8.5 Location Studio

Manages reusable locations.

Information may include:

- Architecture.
- Geography.
- Interior and exterior layouts.

- Lighting.
- Time of day.
- Weather.
- Materials.
- Color palette.
- Reference images.
- Damage states.
- Historical or chronological variations.

Locations should be reusable across scenes.

8.6 Object Studio

Manages reusable visual objects.

Examples:

- Weapons.
- Tools.
- Vehicles.
- Equipment.
- Personal belongings.
- Technology.
- Important story artifacts.

Objects should support reference images, dimensions, materials, appearance variations, and damage states.

8.7 Storyboard Studio

The Storyboard Studio converts scenes into editable panel sequences.

Each scene should contain:

- Scene identifier.
- Source manuscript reference.
- Characters.
- Exact character versions.
- Location.

- Narrative purpose.
- Panel sequence.
- Camera angle.
- Shot type.
- Character positions.
- Actions.
- Expressions.
- Dialogue.
- Captions.
- Background details.
- Art style.
- Approval status.

Storyboards must remain editable before artwork generation.

Multi-character composition

Multi-character scenes are inherently difficult for image generation systems.

PanelForge should provide:

- Character position assignments.
- Relative placement.
- Camera and shot guidance.
- Composition instructions.
- Reference association.
- Manual correction tools.

The application must not promise perfect multi-character placement.

The design should assume that some generated compositions will require manual adjustment or regeneration.

Storyboard history

Support snapshots and lightweight branching.

Users should be able to preserve a working storyboard, experiment with an alternative, and return to the approved version.

Full Git-style version control is not required.

8.8 Art Studio

The Art Studio provides the interface for generating and reviewing artwork.

Initial image backend: ComfyUI.

Functions:

- Generate individual panels.
- Generate variations.
- Regenerate selected assets.
- Inpaint.
- Outpaint.
- Adjust aspect ratios.
- Apply reference images.
- Review generation parameters.
- Compare generations.
- Approve artwork.
- Restore previous versions.

Approved reference comparison

Side-by-side comparison against the approved character reference package must be a first-class feature.

Users should not have to compare a new generation only against the previous generation.

Comparison options should include:

- Current generation versus previous generation.
- Current generation versus approved character portrait.
- Current generation versus approved character reference views.
- Current generation versus approved location or object references.

The goal is to make visual drift easier to identify.

8.9 Comic Layout Studio

Functions:

- Drag-and-drop panel placement.
- Page templates.
- Panel resizing.

- Borders.
- Speech balloons.
- Thought balloons.
- Captions.
- Sound effects.
- Font selection.
- Text positioning.
- Balloon tail direction.
- Reading order.

Two layout formats must be first-class:

1. Traditional comic and graphic novel pages.
2. Vertical webtoon layouts.

Dialogue and captions must remain separately editable.

Text should not be permanently baked into generated artwork.

Lettering and balloon placement are difficult interface problems and must be tested iteratively.

8.10 Continuity Manager

The Continuity Manager checks for inconsistencies across the project.

Potential checks:

- Incorrect character age.
- Wrong character version.
- Clothing inconsistencies.
- Missing or changed equipment.
- Injuries appearing or disappearing.
- Location inconsistencies.
- Chronological conflicts.
- Missing reference material.
- Visual deviations.
- Storyboard and manuscript mismatches.

Warning quality

Continuity checking must prioritize signal over noise.

Warnings should have meaningful severity levels:

- Critical: likely contradiction or major continuity failure.
- Warning: potentially important inconsistency.
- Informational: suggestion or minor discrepancy.

Every warning should explain:

- What appears inconsistent.
- Where the issue occurs.
- Why it matters.
- What the user can do about it.

The user must be able to override warnings.

Avoid repeatedly reporting minor issues that do not materially affect the story.

8.11 Publishing and Export

Initial export formats:

- PDF.
- CBZ.
- PNG.
- JPEG.
- Webtoon image sequences.

Export settings should eventually include:

- Page dimensions.
- Resolution.
- DPI.
- Color profile.
- Bleed.
- Margins.
- Reading direction.
- Cover.

- Metadata.

---

9. Art Style Manager

The Art Style Manager controls the visual identity of a project.

A project should have a canonical default art style profile.

A style profile may contain:

- Name.
- Description.
- Reference images.
- Preferred image models.
- Preferred LoRAs.
- Color palette.
- Lighting characteristics.
- Line-art characteristics.
- Rendering technique.
- Negative prompts.
- Preferred aspect ratios.
- Other generation preferences.

9.1 Style variants

A project may contain multiple styles.

Examples:

- Primary cinematic cyberpunk.
- Muted flashback.
- Surreal dream sequence.
- Technical illustration.
- Historical illustration.

Scenes and individual panels may override the project default.

9.2 Style priority and lock strength

When multiple style profiles and references are active, the system must resolve conflicts predictably.

Initial priority ordering:

1. Explicit panel instructions.
2. Character-specific visual constraints.
3. Scene-specific style override.
4. Project-level style profile.

The system should expose style lock strength or priority settings so users can decide how strongly a style or reference influences generation.

Conflicting instructions should produce a warning when appropriate.

Style controls are guidance for the generation pipeline, not guarantees of perfect visual consistency.

---

10. Resource Manager and Small GPU Mode

The Resource Manager is a core subsystem.

Its purpose is to make PanelForge practical on systems with limited hardware.

The application should treat time as a resource. Users may accept slower generation if the application remains responsive, provides clear progress information, and preserves completed work.

10.1 Hardware detection

At startup, PanelForge should attempt to identify:

- GPU model.
- Available VRAM.
- System RAM.
- CPU.
- Available image-generation backends.
- Available model services.
- Supported monitoring capabilities.

The interface must distinguish between:

- Detected.
- Unavailable.

- Unknown.

It must never represent unavailable telemetry as a zero value or fabricate an estimate.

10.2 Resource profiles

Provide the following profiles:

Safe

- Conservative memory use.
- One generation task at a time.
- Smaller generation sizes where appropriate.
- Conservative model loading.
- Optional cooling intervals.

Balanced

- Reasonable use of available resources.
- Sequential generation by default.
- Limited additional processing when supported.

Performance

- More aggressive use of available resources.
- Higher concurrency only where hardware supports it.
- User awareness of increased resource demand.

Custom

- User-configurable limits.
- Generation resolution.
- Queue concurrency.
- Cooling intervals.
- Other supported backend options.

The application must not overclock hardware or make unsafe fan-control assumptions.

10.3 Small GPU Mode

Small GPU Mode is a first-class operating mode.

It should support practical strategies such as:

- One panel at a time.
- Sequential model loading.
- Lower-resolution drafts.
- Optional upscaling.
- Tiled processing where supported.
- Avoiding simultaneous model loads.
- Releasing unused model resources where supported.
- CPU processing where practical.
- Optional remote generation providers.
- Lower-resource retry options after out-of-memory errors.

The application must not automatically overwrite or downgrade an approved image.

A GTX 1660 Ti class system is an initial practical compatibility target, not a guarantee that every model or workflow will run.

10.4 Thermal awareness

Where supported, the Resource Manager may monitor GPU temperature and other hardware metrics through operating-system or vendor-specific integrations.

Monitoring must be optional and platform-dependent.

The application must support:

- User-configurable soft thresholds.
- User-configurable hard thresholds.
- Pausing new generation tasks.
- Safe handling of active jobs.
- Cooling intervals.
- User notifications.

Electron itself does not directly control GPU temperature, fan speed, or power limits. PanelForge must not promise to do so.

Graceful telemetry failure

Hardware monitoring adapters must fail gracefully.

If temperature, VRAM, or other metrics cannot be obtained:

- Display the value as **Unknown**.

- Do not substitute zero.
- Do not invent an estimate.
- Do not crash or disable the Resource Manager.
- Use conservative scheduling defaults.
- Allow user-configured cooldown intervals.
- Clearly indicate that telemetry is unavailable.

The application must remain fully usable without hardware telemetry.

10.5 Cooling Down state

`Cooling Down` must be an explicit generation task and queue state.

The user must be able to distinguish a cooling pause from:

- A manually paused queue.
- A provider failure.
- A task waiting for resources.
- A completed task.

When cooling down, the interface should explain:

- Why generation has paused.
- Whether a configured interval is running.
- Whether a hardware threshold is being monitored.
- Whether telemetry is unavailable.
- When the next attempt is expected, if that can be determined.

The queue should resume according to the configured policy once the cooling condition has cleared.

---

11. Generation Queue and Task Management

Generation must be organized into small, independent tasks.

Examples:

- Generate character portrait.
- Generate character front view.
- Generate character expression.

- Generate location reference.
- Generate object reference.
- Generate storyboard panel.
- Inpaint a selected region.
- Upscale an image.
- Export a page.

Each task must be independently trackable.

11.1 Task states

The task system should support:

- Queued.
- Preparing.
- Loading Model.
- Generating.
- Post-processing.
- Saving.
- Complete.
- Failed.
- Paused.
- Cancelled.
- Waiting for Resources.
- Cooling Down.

Not every provider will support every intermediate state.

Where the provider cannot report detailed progress, PanelForge should show an indeterminate progress indicator and the best available task status.

It must not invent percentages.

11.2 Progress information

For each task, display:

- Task name.
- Task type.

- Queue position.
- Current stage.
- Elapsed time.
- Estimated remaining time when meaningful.
- Model and backend.
- Resource profile.
- Status.
- Relevant logs or error information.

11.3 Queue controls

Users should be able to:

- Pause.
- Resume.
- Cancel.
- Retry.
- Reorder tasks.
- Change priority.
- Inspect task details.

Batch operations should support resumable execution.

Tasks should be pausable between safe checkpoints where the backend permits it.

11.4 Failure isolation

A failed task must not destroy completed sibling outputs.

If a batch contains ten panels and panel seven fails, the other nine results must remain saved and available.

Only the failed task should require retry unless the user explicitly chooses otherwise.

11.5 Dependency tracking

PanelForge should maintain relationships between source assets and generated results.

For example:

A character reference may be used by several storyboard panels.

If the reference changes, the application should identify affected panels and mark them as potentially outdated.

It must not automatically regenerate them.

The user chooses which dependent assets should be regenerated.

---

12. Generation Provenance and Version History

Every generated image must retain its generation history.

Record, where available:

- Prompt.
- Negative prompt.
- Model name.
- Model hash or identifier.
- LoRAs.
- Seed.
- Exact ComfyUI workflow JSON.
- Node versions.
- Reference image identifiers.
- Generation parameters.
- Output hash.
- Timestamp.
- Backend version.
- Relevant environment information.

The exact workflow should be saved alongside the generated asset or in its associated project metadata.

Reproducibility cannot be guaranteed merely by retaining workflow JSON. Models, custom nodes, runtime versions, and other dependencies may change or become unavailable.

PanelForge should record enough information to help users understand and reproduce the generation environment.

12.1 Version behavior

Every generation creates a new version.

Users must be able to:

- Compare versions.
- Inspect generation parameters.
- Restore an older version.
- Promote a generation to approved.
- Create a variation from an existing generation.
- Preserve an approved version.

Approved artwork must never be silently replaced.

12.2 Versioned project elements

The versioning system should eventually cover:

- Character profiles.
- Character reference packages.
- Locations.
- Objects.
- Art style profiles.
- Storyboards.
- Page layouts.
- Generated artwork.

---

13. Security and Reliability

The application must use secure Electron defaults.

Requirements:

- Context isolation enabled.
- Node integration disabled in the renderer.
- Sandboxing enabled where compatible.
- Restricted preload API.
- IPC allowlist.
- Argument validation.

- Controlled filesystem access.
- Safe path handling.
- Centralized error handling.
- Application logging.

The application should recover gracefully from:

- AI provider disconnection.
- Failed generation.
- Interrupted tasks.
- Database locking.
- Missing files.
- Corrupted asset references.
- Application shutdown during generation.

Existing projects must remain accessible even when AI services are unavailable.

---

14. Technology Stack

The initial technology choices are:

Area	Technology
Desktop application	Electron
Language	TypeScript
Runtime	Node.js
Interface	HTML, CSS, JavaScript/TypeScript
UI framework	Lightweight framework if it materially improves development
Database	SQLite
Database access	Drizzle ORM or equivalent
Local LLM	LM Studio
Image generation	ComfyUI
Image processing	Sharp
Page composition	SVG and Canvas
PDF generation	Suitable PDF library
Document parsing	DOCX, EPUB, PDF parsers
Testing	Vitest and Playwright
Packaging	Electron Forge
Source control	Git

The application should be developed on Linux Mint first.

Windows compatibility should be considered in architecture and packaging decisions from the beginning, but Windows-specific implementation must not delay the initial Linux foundation.

The project must not be unnecessarily constrained by a framework-light philosophy. A small UI framework or state-management solution is acceptable if it makes the application easier to build and maintain.

---

15. Development Roadmap

Development is divided into stages.

Each stage contains milestones, and each milestone contains small implementation tasks.

The development order is mandatory.

Stage 0 and Stage 1 must be completed before implementing Character Studio or real ComfyUI integration.

Interface placeholders and mock services may be used where necessary, but they must not become excuses to begin implementing later systems prematurely.

Stage 0: Application Foundation

Goal: Establish a reliable Electron application with secure architecture, configuration, logging, and database infrastructure.

S0.1: Repository and package baseline

Goal: Create the project repository and initial package configuration.

Deliverables:

- Git repository.
- Package configuration.
- Initial directory structure.
- Basic development scripts.
- README.

Acceptance:

- Dependencies install successfully.
- Development scripts execute.
- Repository contains no generated build artifacts or secrets.

S0.2: Electron application launch

Goal: Create the initial desktop application.

Deliverables:

- Electron main process.
- Basic BrowserWindow.
- Initial renderer page.

Acceptance:

- Application launches on Linux Mint.
- Window opens reliably.
- Application closes cleanly.

S0.3: Secure BrowserWindow configuration

Goal: Establish secure renderer defaults.

Deliverables:

- Context isolation.
- Disabled Node integration.
- Sandboxing where compatible.
- Restricted preload bridge.

Acceptance:

- Renderer cannot directly access Node.js.
- Application security settings are documented and tested.

S0.4: IPC foundation

Goal: Create a controlled communication channel.

Deliverables:

- IPC allowlist.
- Typed request and response structures.
- Argument validation.
- Centralized IPC registration.

Acceptance:

- Only registered methods can be called.

- Invalid arguments are rejected.
- Errors are returned safely.

S0.5: Safe diagnostic operation

Goal: Verify the renderer-to-main communication path.

Deliverables:

- One diagnostic IPC operation.
- Preload API method.
- Renderer test interface.

Acceptance:

- Renderer successfully requests diagnostic information.
- No unrestricted Node.js access is required.

S0.6: Configuration and user paths

Goal: Centralize application configuration.

Deliverables:

- Configuration service.
- User data path handling.
- Application settings location.
- Default configuration.

Acceptance:

- No hardcoded machine-specific paths are required.
- Configuration can be read and updated safely.

S0.7: Logging and error handling

Goal: Establish useful diagnostics.

Deliverables:

- Central logging service.
- Error handling.
- Size-limited or rotating logs.
- Consistent error reporting.

Acceptance:

- Errors are recorded.
- Logs remain within configured size limits.
- Sensitive information is not unnecessarily logged.

S0.8: SQLite foundation

Goal: Establish database infrastructure.

Deliverables:

- SQLite connection service.
- Initial schema.
- Migration mechanism.
- WAL mode.
- Foreign key enforcement.
- Busy timeout.
- Serialized write service.

Acceptance:

- Database initializes successfully.
- Migrations execute.
- Data persists across restarts.
- Basic concurrent access tests do not cause uncontrolled write failures.

S0.9: Application lifecycle infrastructure

Goal: Provide reliable startup and shutdown behavior.

Deliverables:

- Startup sequence.
- Shutdown sequence.
- Resource cleanup hooks.
- Basic application lifecycle events.

Acceptance:

- Application starts and closes without leaving avoidable resources behind.

S0.10: Keyboard shortcuts and navigation foundation

Goal: Establish basic keyboard accessibility.

Deliverables:

- Shortcut registration.
- Basic navigation structure.
- Shortcut documentation.

Acceptance:

- Core navigation can be used without relying exclusively on mouse interaction.

S0.11: Initial packaging

Goal: Produce an installable or runnable Linux build.

Deliverables:

- Linux packaging configuration.
- Development build.
- Initial packaged build.

Acceptance:

- Packaged application launches independently of the development environment.

S0.12: Stage 0 verification

Goal: Verify the complete foundation.

Deliverables:

- Smoke tests.
- Updated README.
- Known limitations.
- Stage 0 completion report.

Acceptance:

- All Stage 0 tests pass.
- No critical security or startup issues remain.
- Linux build is repeatable.

Stage 0 stop condition: Do not proceed until the application shell, security boundary, configuration, logging, SQLite foundation, and Linux build are reliable.

Stage 1: Project Management and Portable Storage

Goal: Establish the project format, storage conventions, asset registry, and backup system.

S1.1: Project folder format

Deliverables:

- Project folder specification.
- Project version marker.
- Initial project metadata structure.

Acceptance:

- A valid project can be identified and version-checked.

S1.2: Project metadata schema

Deliverables:

- Project database tables.
- Project identifier.
- Name and description fields.
- Creation and modification timestamps.

Acceptance:

- Project metadata persists correctly.

S1.3: Basic project operations

Deliverables:

- Create.
- List.
- Open.
- Close.
- Rename.
- Archive.

Acceptance:

- Each operation works independently.
- Invalid project operations return useful errors.

S1.4: Relative asset path service

Deliverables:

- Relative path resolution.
- Path normalization.
- Path traversal protection.

Acceptance:

- Valid project paths resolve correctly.
- Unsafe paths are rejected.

S1.5: Asset registry

Deliverables:

- Asset records.
- Asset type metadata.
- File hashes.
- Import and copy operations.

Acceptance:

- Imported assets are registered and retrievable.

S1.6: Asset integrity checking

Deliverables:

- Missing asset detection.
- Hash verification.
- Integrity report.

Acceptance:

- Missing or modified assets are reported accurately.

S1.7: Atomic metadata operations

Deliverables:

- Safe metadata writes.
- Recovery from interrupted writes.
- Basic autosave mechanism.

Acceptance:

- Interrupted writes do not leave project metadata partially corrupted.

S1.8: Backup and snapshot system

Deliverables:

- Snapshot creation.
- Backup creation.
- Restore operation.
- Verification.

Acceptance:

- A snapshot can be restored and verified without damaging the original project.

S1.9: Project portability test

Deliverables:

- Project copy/move test.
- Path verification.
- Asset integrity verification.

Acceptance:

- A copied project opens on a different directory path without relying on the original machine's absolute paths.

S1.10: Basic project interface

Deliverables:

- Project list.
- Create project interface.
- Open project interface.
- Basic project information.

Acceptance:

- Users can manage projects without direct database interaction.

S1.11: Stage 1 test fixture

Deliverables:

- Sample project.
- Automated tests.

- Project format documentation.

Acceptance:

- A sample project can be created, copied, reopened, backed up, restored, and verified.

Stage 1 stop condition: Do not begin Character Studio or real ComfyUI integration until project creation, storage, asset tracking, portability, and backups are reliable.

Stage 2: AI Orchestration and Resource Management

Goal: Establish the infrastructure for AI providers and background tasks.

This stage must be divided into small tasks before implementation.

Initial task groups:

- Provider interface definitions.
- LM Studio connection adapter.
- ComfyUI connection adapter.
- Provider health checks.
- Persistent task records.
- Task queue.
- Task state transitions.
- Pause and cancellation.
- Retry handling.
- Progress reporting.
- Resource profiles.
- Hardware detection.
- Telemetry adapter interface.
- Graceful telemetry failure.
- Cooling Down state.
- Queue persistence and recovery.
- Small GPU Mode.
- Mock provider testing.

Acceptance criteria must verify that tasks can be created, persisted, executed, interrupted, resumed where supported, and recovered after application restart.

Real generation must remain independently testable from the user interface.

Stage 3: Manuscript Analysis and Art Style Profiles

Goal: Import written material, analyze it, and establish visual style definitions.

Task groups:

- TXT and Markdown import.
- DOCX import.
- EPUB import.
- PDF import.
- Chapter extraction.
- Scene extraction.
- Source text relationships.
- Character extraction.
- Location extraction.
- Object extraction.
- Event and chronology extraction.
- Canon versus interpretation classification.
- Art Style Manager.
- Style profile persistence.
- Style variants.
- Style priority resolution.

Acceptance:

- Original manuscripts remain preserved.
- Extracted information links back to source material.
- Art styles can be created, saved, and applied to scenes.

Stage 4: Character Studio

Goal: Create character profiles, visual references, and version management.

Task groups:

- Character profile CRUD.
- Character version records.
- Reference asset association.
- Reference completeness scoring.
- Minimum Viable Reference Set.
- Character reference generation.
- Generation approval.
- Character version binding.
- Reference comparison.
- Character history.

Acceptance:

- Characters can have multiple visual versions.
- Every panel can identify the exact character version it uses.
- Incomplete references produce actionable warnings.

Stage 5: Location and Object Studio

Goal: Manage reusable visual references for environments and objects.

Task groups:

- Location profiles.
- Location reference management.
- Location variations.
- Object profiles.
- Object reference management.
- Object variations.
- Asset history.
- Generation integration.

Acceptance:

- Locations and objects can be independently created, edited, referenced, and regenerated.

Stage 6: Storyboard Studio

Goal: Convert scenes into editable panel sequences.

Task groups:

- Scene records.
- Panel records.
- Character and version assignment.
- Location assignment.
- Camera and shot definitions.
- Character positioning.
- Dialogue and captions.
- Scene style overrides.
- Panel instructions.
- Storyboard snapshots.
- Lightweight branching.
- Composition review.

Acceptance:

- A storyboard can be created and edited without generating artwork.
- Every panel identifies its source scene and assigned character versions.
- Storyboard changes do not silently modify approved artwork.

Stage 7: Art Studio

Goal: Generate, review, compare, and approve individual artwork assets.

Task groups:

- ComfyUI workflow integration.
- Workflow templates.
- Prompt generation.
- Negative prompt handling.
- Reference conditioning.
- Individual panel generation.

- Variations.
- Inpainting.
- Outpainting.
- Side-by-side comparison.
- Approved reference comparison.
- Generation provenance.
- Version history.
- Approval and restoration.
- Dependency tracking.

Acceptance:

- Individual panels can be generated and regenerated independently.
- Generation history is retained.
- Approved work is preserved.
- Workflow provenance is stored.

Stage 8: Comic Layout Studio

Goal: Assemble approved artwork into finished pages.

Task groups:

- Page creation.
- Traditional page layouts.
- Webtoon layouts.
- Panel placement.
- Panel resizing.
- Borders.
- Balloon creation.
- Balloon positioning.
- Tail direction.
- Dialogue editing.
- Caption editing.

- Sound effects.
- Reading order.
- Layout snapshots.

Acceptance:

- A page can be assembled without modifying the original artwork.
- Dialogue and captions remain editable.
- Both traditional and vertical formats work.

Stage 9: Continuity and Quality Assurance

Goal: Detect meaningful inconsistencies.

Task groups:

- Character version checks.
- Age checks.
- Clothing checks.
- Equipment checks.
- Injury checks.
- Location checks.
- Chronology checks.
- Reference completeness checks.
- Severity classification.
- Warning suppression and overrides.
- Actionable recommendations.

Acceptance:

- Major inconsistencies are reported.
- Minor issues do not overwhelm the user.
- Warnings identify the affected scene or panel.

Stage 10: Publishing and Export

Goal: Produce finished deliverables.

Task groups:

- PDF export.
- CBZ export.
- PNG export.
- JPEG export.
- Webtoon sequence export.
- Page dimensions.
- Resolution settings.
- DPI.
- Margins and bleed.
- Metadata.
- Export validation.

Acceptance:

- Exported documents preserve the approved page layout.
- Text remains readable.
- Output files pass basic integrity checks.

Stage 11: Advanced Features

Deferred features may include:

- Headless command-line operation.
- Batch processing.
- Background daemon.
- Advanced continuity checks.
- LoRA training.
- Advanced reference conditioning.
- Animation.
- Video.
- Voice.
- Localization.

- Cloud synchronization.
- Distributed generation.
- Advanced print-production features.

The Core Engine should be designed to support future headless operation, but a separate CLI or daemon is not required for the initial release.

Possible future commands:

```
panelforge-cli project list
panelforge-cli analyze chapter-05
panelforge-cli continuity check chapter-05
panelforge-cli generate storyboard-05
panelforge-cli export chapter-05 --format pdf
```

---

16. Development Methodology

The development process is as important as the technical architecture.

PanelForge must be built incrementally.

16.1 Three-level task structure

Development uses:

Stage → Milestone → Task

A stage represents a major subsystem.

A milestone represents a functional piece of that subsystem.

A task represents a small implementation unit that can be completed and tested independently.

16.2 Task requirements

Every implementation task must contain:

- Task identifier.
- Goal.
- Scope.
- Deliverable.
- Dependencies.
- Acceptance criteria.
- Stop condition or explicit exclusions where appropriate.

A task should normally fit within one or two focused coding sessions.

If a task is too large, split it before implementation.

16.3 Implementation Task Register

Maintain a task register with:

- Task ID.
- Description.
- Status.
- Dependencies.
- Test status.
- Notes.
- Completion date.

Suggested statuses:

- Not Started.
- Ready.
- In Progress.
- Blocked.
- Testing.
- Complete.
- Deferred.

Only tasks whose dependencies are satisfied should be marked Ready.

16.4 Definition of task completion

A task is not complete merely because code has been written.

It must:

1. Compile or execute successfully.
2. Pass its acceptance test.
3. Include relevant automated tests where practical.
4. Handle expected errors.
5. Avoid breaking previously completed functionality.

6. Be documented where its behavior or interface requires documentation.

16.5 No premature subsystem development

Do not start implementing later-stage functionality simply because it is interesting or visually rewarding.

In particular:

- No real Character Studio implementation before Stage 0 and Stage 1 are complete.
- No real ComfyUI integration before the foundation and project storage are complete.
- No large-scale storyboard or layout implementation before the underlying data structures are established.

Mocks and interface prototypes are acceptable when they help validate an architectural decision.

They must not become uncontrolled parallel development.

16.6 Testing discipline

Each subsystem should have:

- Unit tests.
- Integration tests where appropriate.
- Manual acceptance tests for user-facing behavior.
- Regression tests for previously completed functionality.

A completed stage should have a documented verification result.

16.7 Source control

Git is mandatory for PanelForge source code.

Use commits to preserve working milestones.

User projects do not require Git. Their history and recovery mechanisms are handled by PanelForge's own project snapshot and versioning systems.

---

17. Milestones

Milestone A: Reliable Desktop Foundation

Includes Stages 0–2.

Deliverable:

- Working Linux desktop application.
- Secure Electron architecture.
- Reliable SQLite storage.
- Project infrastructure.
- AI provider interfaces.
- Persistent task queue.
- Resource Manager foundation.

Milestone B: Manuscript to Approved Artwork

Includes Stages 3–7.

Deliverable:

- Import a manuscript.
- Analyze its contents.
- Establish art styles.
- Create character references.
- Create locations and objects.
- Build storyboards.
- Generate and approve individual panels.

Milestone C: Finished Comic Production

Includes Stages 8–10.

Deliverable:

- Assemble artwork.
- Add lettering.
- Check continuity.
- Export finished comics and webtoons.

Milestone D: Advanced Production

Includes Stage 11.

Deliverable:

- Additional production automation and advanced capabilities.

Milestones are not permission to skip intermediate acceptance tests.

---

18. MVP Definition

The initial usable product should demonstrate one complete workflow rather than attempt to implement every planned feature.

The MVP should support:

1. Create a project.
2. Import a manuscript.
3. Identify characters and scenes.
4. Create or edit character profiles.
5. Create character reference images.
6. Establish a project art style.
7. Create a storyboard.
8. Generate individual panels.
9. Review and approve artwork.
10. Assemble a simple comic page.

The initial MVP does not require:

- LoRA training.
- Advanced continuity scoring.
- Animation.
- Video.
- Cloud synchronization.
- Advanced collaboration.
- Complex print-production workflows.

The MVP should demonstrate the core production pipeline reliably.

---

19. Acceptance Test: Nemesis Star, Broken Wings

The first major acceptance test will use Chapter 5, **Hunger Wins**, from *Nemesis Star: Broken Wings*.

The chapter includes an eight-year-old Robby and his imaginary companion ARKNAD.

The acceptance test should verify that PanelForge can preserve the narrative and visual requirements of a specific piece of fiction.

19.1 Character requirements

Robby must be represented as an eight-year-old child.

The system must not accidentally use an older version of the character.

ARKNAD is Robby's imaginary space-pirate OmniPad companion.

Its design should communicate a stronger space-pirate identity rather than defaulting to a generic tablet or generic assistant appearance.

19.2 Scale and perspective

The Guardians are enormous.

Their exaggerated size should be retained.

Because the chapter is presented from a child's perspective, the exaggerated scale may be appropriate even if the resulting imagery is not a literal objective measurement.

The system should distinguish intended visual exaggeration from an accidental continuity error.

19.3 Setting

Ravenholt must be represented consistently.

Locations should be reusable across the relevant panels.

19.4 Narrative and chronology

The generated storyboard must preserve:

- The chapter's sequence of events.
- Character presence.
- Dialogue.
- Emotional context.

- Important actions.
- Relevant visual details.

The system must not invent events to fill gaps without identifying them as suggestions.

19.5 Exact character version binding

The acceptance test must explicitly record the exact character version assigned to each panel.

For example, an eight-year-old Robby reference must be assigned to the appropriate panels rather than an adult or older character version.

19.6 Generation and hardware behavior

The test should verify:

- Individual panel generation.
- Independent regeneration.
- Queue progress visibility.
- Preservation of successful outputs.
- Small GPU Mode behavior.
- Resource monitoring when available.
- Unknown telemetry handling when monitoring is unavailable.
- Cooling Down state behavior.

19.7 Acceptance outcome

The test is successful when a small sequence of approved panels can be generated, reviewed, assembled, and exported while preserving the chapter's intended visual and narrative continuity.

The objective is not perfect automated comic production.

The objective is a usable production workflow in which the creator remains in control.

---

20. Risks and Mitigations

Risk	Mitigation
Project scope becomes too large	Small implementation tasks and strict stage sequencing
Limited GPU memory	Small GPU Mode, sequential generation, lower-resource profiles
Hardware telemetry unavailable	Unknown state, graceful fallback, conservative scheduling

Thermal concerns	Optional monitoring, cooling intervals, conservative defaults
AI character inconsistency	Versioned references, exact panel binding, comparison tools
Multi-character generation failures	Composition guidance and manual correction
Story analysis invents facts	Canon classification and source traceability
Database locking	WAL, serialized writes, busy timeout and retry handling
Project portability failures	Relative paths and integrity verification
Generation failures	Persistent task state and partial output preservation
Visual style drift	Style profiles, priority resolution and approved references
Continuity warning fatigue	Severity levels and actionable warnings
Lettering interface complexity	Iterative design and focused acceptance testing
Workflow reproducibility problems	Exact workflow and dependency provenance

---

21. Repository Structure

The initial repository should follow a modular structure similar to:

```
panelforge/
├── src/
│   ├── main/
│   │   ├── electron/
│   │   ├── security/
│   │   └── lifecycle/
│   ├── preload/
│   ├── renderer/
│   │   ├── views/
│   │   ├── components/
│   │   ├── styles/
│   │   └── navigation/
│   ├── core/
│   │   ├── projects/
│   │   ├── tasks/
│   │   ├── events/
│   │   ├── configuration/
│   │   ├── logging/
│   │   └── resources/
│   ├── engines/
│   │   ├── llm/
│   │   ├── image/
│   │   ├── analysis/
│   │   └── export/
│   ├── modules/
│   │   ├── manuscripts/
│   │   ├── characters/
│   │   ├── locations/
│   │   ├── objects/
│   │   ├── styles/
│   │   ├── storyboards/
│   │   ├── artwork/
│   │   ├── continuity/
│   │   └── publishing/
│   ├── database/
```

```
│   │   ├── schema/
│   │   └── migrations/
│   └── shared/
│       ├── types/
│       └── interfaces/
├── workflows/
│   └── comfyui/
├── resources/
├── tests/
├── scripts/
├── package.json
└── README.md
```

The structure may be adjusted as implementation reveals better boundaries.

Avoid creating empty modules solely to make the directory tree look complete.

Only introduce a subsystem when it is needed by the current development stage.

---

22. Definition of Done

PanelForge is not considered ready for initial release simply because it can generate images.

The initial usable product must demonstrate:

- Reliable project creation and storage.
- Manuscript import.
- Character profile management.
- Character reference management.
- Art style management.
- Editable storyboards.
- Individual artwork generation.
- Generation history.
- Approved reference comparison.
- Independent regeneration.
- Task queue visibility.
- Small GPU Mode.
- Graceful handling of unavailable hardware telemetry.
- Cooling Down state.
- Basic page composition.

- Editable dialogue and lettering.
- Export of finished work.
- Recovery and backup functionality.

The system must remain useful when AI services are unavailable.

The user must retain control over every approved asset.

---

23. Immediate Development Starting Point

Development begins with Stage 0 only.

The first implementation session should establish:

1. Repository and package baseline.
2. Electron application launch.
3. Secure BrowserWindow configuration.
4. Restricted preload bridge.
5. Basic IPC communication.

Subsequent tasks should be completed and tested individually.

SQLite, configuration, logging, and packaging follow within Stage 0.

No Character Studio work. No real ComfyUI integration. No premature image-generation experiments inside the application.

The first objective is a reliable desktop application foundation.

Once Stage 0 is accepted, proceed to Stage 1.

Once Stage 1 is accepted, proceed to Stage 2.

PanelForge will be built as a series of working, testable pieces rather than one enormous unfinished application.

Final Product Principle

PanelForge should make serious illustrated storytelling accessible to creators who do not have expensive hardware, large production teams, or specialized technical knowledge.

It should allow a creator to build a story one character, one reference, one panel, and one page at a time.

Generation may be slow. Some results will be imperfect. Some compositions will need manual work.

That is acceptable.

What is not acceptable is losing approved work, hiding what the application is doing, forcing unnecessary regeneration, or making the software unusable because the user's GPU isn't the newest model.

Work with the hardware you have. Keep control of your creative work. Build one piece at a time.

End of Specification.